

Hexagonal, Onion & Clean Architecture

One big idea, three names: put the domain at the center.

Why Not Just Layered?

Layered architecture couples your domain to infrastructure — the business logic depends on the database. These styles fix that, all the same way.

Layered problem

- Domain depends on persistence
- Hard to test without the database
- Frameworks & DB drive the design

The cure

- Domain depends on nothing external
- Infrastructure depends on the domain
- Test the core in complete isolation

Learning Objectives

- Explain the one idea behind hexagonal, onion and clean architecture.
- Use ports and adapters to keep the domain independent.
- Read the concentric model and the dependency rule.
- Weigh the benefits against the extra indirection they cost.

Invert the Dependencies

It's the Dependency Inversion Principle applied to a whole system: high-level policy (the domain) must not depend on low-level detail (the database).

**The domain sits at the center and depends on nothing.
Databases, frameworks and UIs sit at the edges — and depend on the domain.**

Dependencies always point inward, toward the domain.

Section 01

Hexagonal — Ports & Adapters

The domain talks to the world only through ports.

Ports and Adapters

The domain decides every port. Infrastructure only implements them.

Port — the domain decides

- Declared by the domain, for the domain
- in = driving · out = driven
- OrderUseCase · OrderRepository

Adapter — infrastructure obeys

- Implements a port; never defines one
- The Gang of Four's Adapter pattern
- OrderController · InMemoryOrderRepository

Reverse that sentence and you have layered architecture again.

In and Out, All the Way Down

in and out name which side starts the conversation — in the ports and in the adapters alike.

tree

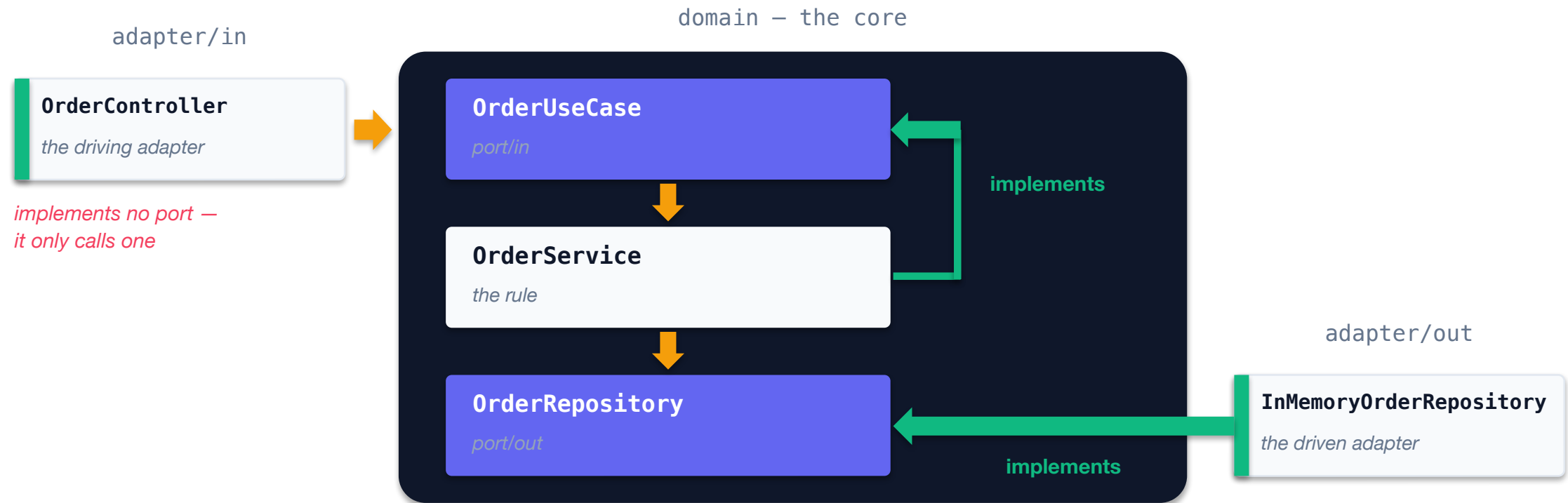
```
domain/  
  port/  
    in/      OrderUseCase      driving  <- implemented by OrderService  
    out/     OrderRepository    driven    <- implemented by InMemoryOrderRepo  
    OrderService                implements port/in, calls port/out  
  
adapter/  
  in/      OrderController  calls OrderUseCase (it implements no port)  
  out/     InMemoryOrderRepo implements OrderRepository
```

Evans had the out side in 2003 — a domain-owned Repository. Hexagonal's addition is the in side.

Kept small on purpose — Tom Hombergs' BuckPal goes further; see the README.

Who Calls, and Who Implements

The same five files as page 7. What a tree cannot show is which way each arrow points.

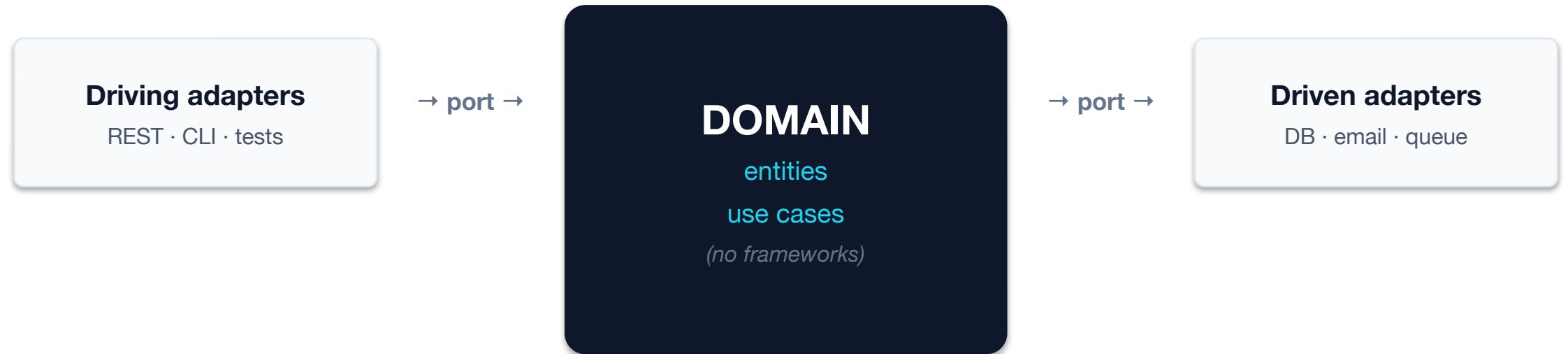


control flow in through the port at the top, then down through the core

implements always points at the port, never away from it

Both adapters depend on the core. The core depends on neither — that is the inversion.

The Domain at the Center



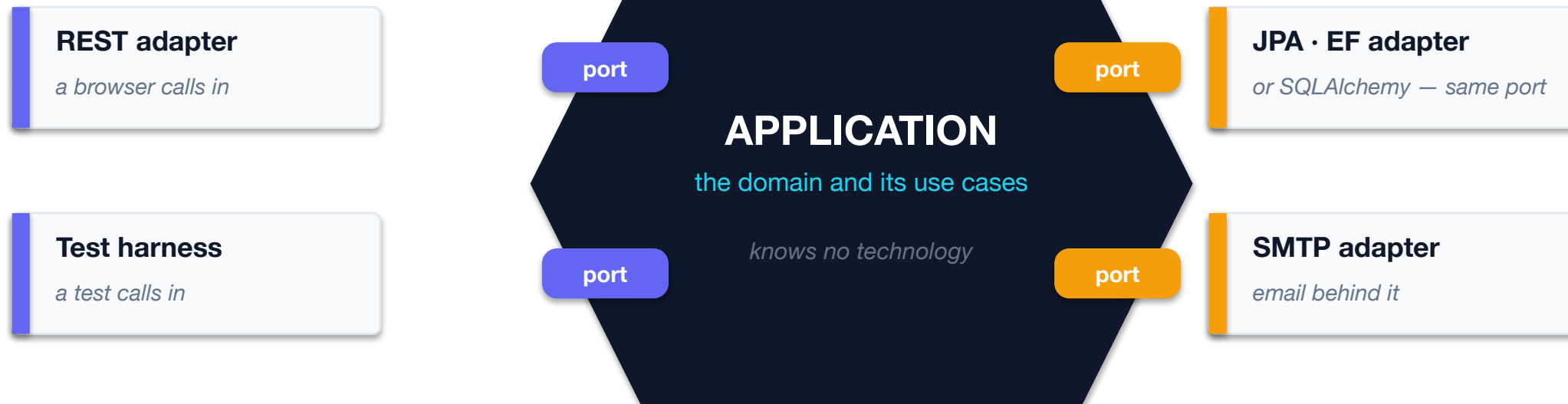
The outside drives the domain through ports; the domain drives the outside through ports. Either edge can be swapped without touching the core.

Why It's a Hexagon

Cockburn drew it with sides so there would be somewhere to put the ports.

PRIMARY · *driving*

SECONDARY · *driven*



“The hexagon is not a hexagon because the number six is important, but rather to allow the people doing the drawing to have room to insert ports and adapters as they need, not being constrained by a one-dimensional layered drawing.”

— Alistair Cockburn, alistair.cockburn.us/hexagonal-architecture

The Same Request, Ports and Adapters

Flow*In through a port, out through a port*

```
POST /orders                                (adapter/in)
  → OrderController.place(req)
    → OrderUseCase.place(order)             (domain/port/in – an interface)
      → OrderService.place(order)           (domain: rejects a blank order)
        → OrderRepository.save()            (domain/port/out – an interface)
          → InMemoryOrderRepository         (adapter/out)
← 201 Created                               (back out through the adapter)
```

Same request, same three classes as page 7 of Layered Architecture. What changed is the two interfaces it now passes through — and that both belong to the domain, not to the layer below.

adapter/in · domain/port/in · domain/OrderService · domain/port/out · adapter/out

github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

The Adapter Really Is Adapter

Borrowed, not invented. The Gang of Four's Adapter converts one interface into another a client already expects — which is what an edge does, once real technology sits behind it.

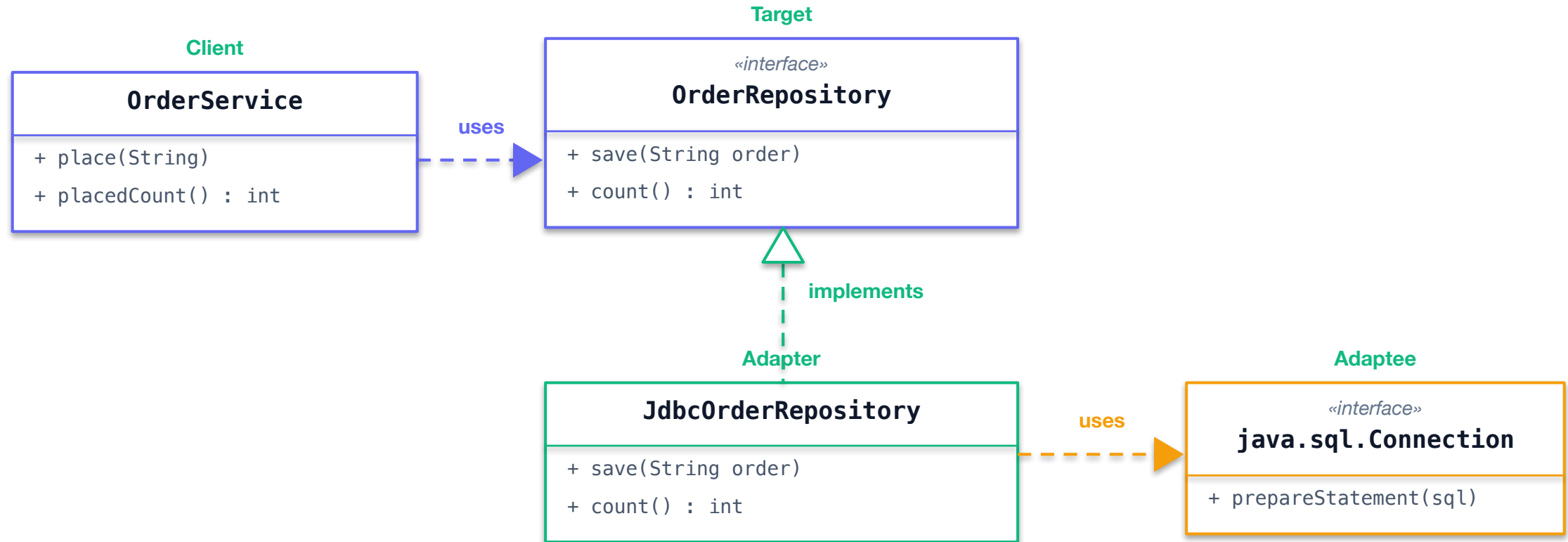
GoF role	Driven side — the domain reaches out	Driving side — the world reaches in
Client	The domain — OrderService	The web framework
Target	The port — OrderRepository	The framework's handler signature
Adaptee	JDBC, SMTP, an HTTP client	The port — OrderUseCase
Adapter	JdbcOrderRepository	OrderController

Read each column downward. The same port is a Target on one side and an Adaptee on the other.

An adapter becomes the GoF Adapter once something foreign sits behind the port. Page 14 is what it has to convert.

Adapter, as a Class Diagram

The driven column of page 12, drawn. Two relationships, and UML draws each one differently.



Dashed + hollow triangle = realization: implements an interface. Solid + triangle would be generalization: extends a class.

OrderService names no technology. The only class that knows JDBC exists is the adapter.

What Makes the Interfaces Incompatible

Three things differ at every edge. The adapter is where all three are resolved — which is why it is the one place boilerplate is allowed to live.



Vocabulary

The domain says `save(Loan)`.
Postgres says `INSERT` and columns. The adapter maps the types, both directions.



Failure

`SQLException`, a 404, a timeout — none of these are domain ideas. The adapter turns them into the domain's own errors, or into a status code on the way back.



Protocol

Connections, transactions, retries, JSON, HTTP verbs. All of it stays out here, so the domain never learns the technology exists.

The Domain Owns the Interface

Domain (inside)

```
// port – defined by the domain
// it names no technology, and could not
interface OrderRepository {

    void save(String order);

    int count();
}
```

Adapter (edge)

```
// adapter – implements the port
final class InMemoryOrderRepository
    implements OrderRepository {
    private final List<String> orders;
    public void save(String o) {
        orders.add(o);
    }
}
```

The domain depends only on OrderRepository. The adapter depends on the domain — not the reverse.

Swap in a Postgres adapter and the domain never notices — that sentence is the Javadoc on the real class.

Two Ways to Get What the Domain Needs

A rule needs a fact only infrastructure holds. Either it reaches out through a port of its own, or the application service fetches the fact and hands it in.

Port inside the rule

```
// the rule calls out, mid-decision
// it needs a LoanRepository to answer
class Loan {
    boolean canRenew(LoanRepository repo) {
        return repo.activeCountForMember(id) < 5;
    }
}
```

Fact handed in

```
// the rule is told what it needs
// the service looked it up already
class Loan {
    boolean canRenew(int activeLoans) {
        return activeLoans < 5;
    }
}
```

Is what the rule needs knowable before the decision starts?

If it is, hand it in — the application service fetches, the domain decides.

Both are still ports and adapters — on the right the port moved up to the application ring (page 22).

Testing the rule on the left needs a stub; on the right it needs an int. That is the whole argument.

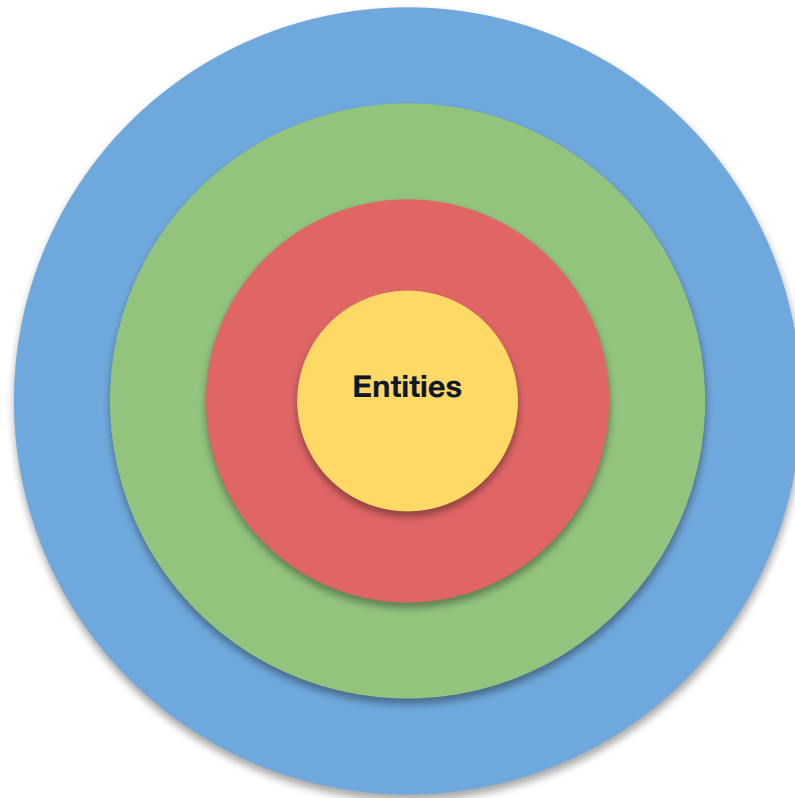
Section 02

Onion & Clean

The same idea, drawn as concentric rings.

The Drawing Everyone Has Seen

Hexagonal draws the boundary. Onion and Clean draw what is inside it — Martin's own diagram, redrawn.



Frameworks & Drivers

Web · UI · DB · devices · external interfaces

Interface Adapters

controllers · gateways · presenters

Application Business Rules

use cases — what this application does

Enterprise Business Rules

entities — true with or without this app

The Dependency Rule: source code dependencies point only inward.

Redrawn from Robert C. Martin's original → blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

Why Inward and Not Outward

The rings are not sorted by importance, and not really by how often they change. They are sorted by **whose reason** they change for.



Changes for the business

A new discount rule, a tax change, a new kind of order. This churns constantly — it is the work you are paid for, and it should be cheap.



Changes for other reasons

A vendor bumps a major version. A screen is redesigned. REST becomes gRPC. None of it is your business changing.



So the arrow points inward

...so a change of the second kind can never force a change of the first. Your rules still change every sprint — never because Hibernate did.

This is chapter 04's Single Responsibility Principle at architectural scale — one reason to change, applied to a whole ring.

 Zoom out

Three Names, One Principle

Style	Center	Mechanism
Hexagonal	Domain	Ports & adapters
Onion	Domain model	Concentric layers, depend inward
Clean	Entities	The dependency rule

All three: the domain at the center, dependencies pointing inward, infrastructure at the edges. Pick the vocabulary your team likes.

What Each One Adds

They are not three inventions. Each takes the one before it and **adds something specific** — and the third adds less than its fame suggests.

Hexagonal · 2005

Symmetry. The application meets the world only through ports, and the driving side is treated exactly like the driven one. Silent about the inside.

Onion · 2008

Structure inside the core. Named rings — domain model, domain services, application services — plus the rule that all coupling points inward.

Clean · 2012

Vocabulary, mostly. One Dependency Rule, entities split from use cases, and named crossing points: controllers, presenters, gateways.

Martin describes his own diagram as an attempt at integrating all these architectures into a single actionable idea. Read that as synthesis, not a fourth invention.

What an Application Service Does

Evans named this layer in Domain-Driven Design; Onion and Clean both kept it.

“This layer is kept thin. It does not contain business rules or knowledge, but only coordinates tasks and delegates work to collaborations of domain objects in the next layer down.” — Evans, ch. 4

■ mechanism ■ delegation to the domain

Java*appservice/application/PlaceOrderService.java*

```
public void place(String text) {  
    transactions.begin();  
    try {  
        repository.save(Order.of(text));  
        transactions.commit();  
    } catch (RuntimeException e) {  
        transactions.rollback();  
        throw e;  
    }  
}
```

No third color — every line is a mechanism or a delegation, and not one is a business rule.

*The day an **if** about orders appears here, a rule has left the domain. Compare page 10 of Layered Architecture.*

And Vertical Slice Architecture?

Jimmy Bogard, 2018. Organize the code by feature — everything one request needs in one place — instead of by technical role.

A different question

Hexagonal says what may depend on what. Slices say how to group the files. You can slice a hexagonal codebase by feature and break no rule.

Where it really disagrees

Bogard removes the uniform barriers — each slice decides for itself what abstractions it needs. Clean applies the same structure everywhere.

What it costs

A rule is traded for judgment. He says so: it needs a team that can see a smell and refactor to the pattern. That is chapters 03 and 05.

Martin said half of it in 2011 — the top-level folders should shout the use cases, not the framework.

Screaming Architecture, 2011. The other half — each slice deciding for itself — is the real departure.

Read Them First-Hand

Three styles, three authors — all still online, and all short.

Hexagonal *Alistair Cockburn, 2005*

alistair.cockburn.us/hexagonal-architecture *The original page — “Ports and Adapters”*

Onion *Jeffrey Palermo, 2008–2013 — four parts*

jeffreypalermo.com/2008/07/the-onion-architecture-part-1 *Part 1 — the rings, and why the database is not the center*

jeffreypalermo.com/2008/07/the-onion-architecture-part-2 *Part 2 — what goes in each ring*

jeffreypalermo.com/2008/08/the-onion-architecture-part-3 *Part 3 — the code, and the cost*

jeffreypalermo.com/2013/08/onion-architecture-part-4-after-four-years *Part 4 — After Four Years: what he would change*

Clean *Robert C. Martin, blog.cleancoder.com*

blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html *The Clean Architecture — the famous diagram*

blog.cleancoder.com/uncle-bob/2011/09/30/Screaming-Architecture.html *Screaming Architecture — what a source tree shouts*

blog.cleancoder.com/uncle-bob/2016/01/04/ALittleArchitecture.html *A Little Architecture — becoming an architect*

What You Gain — and Pay

Gain

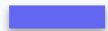
- Test the domain with zero infrastructure
- Swap DB, framework or UI freely
- Business rules survive tech churn

Pay

- More interfaces and indirection
- Mapping between layers (boilerplate)
- Overkill for a small, simple app

This Is Phase 1, Scaled Up

Nothing here is new — it's the principles you already know, applied to whole systems.



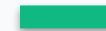
DIP

Depend on abstractions —
now at architectural scale.



Low coupling

The domain is decoupled from
all infrastructure.



Testability

The core is pure, so it's trivially
testable.

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 9 tests here

`hexagonal/domain/port/`

in = driving, out = driven

`hexagonal/adaptor/in + out/`

the two edges

`appservice/`

the same, + a tx-owning app service

C# / .NET

xUnit · 9 tests here

`Architecture.cs`

domain.port + adaptor, in/out

`AppService.cs`

an application service owns the tx

in/out are C# keywords — hence

inbound/outbound

Python

pytest · 9 tests here

`hexagonal/domain/port/`

inbound.py · outbound.py

`hexagonal/adaptor/`

inbound.py · outbound.py

`appservice/`

application/service.py owns the tx

github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

A transaction is a mechanism, not a rule. That is why appservice declares it outside the domain.

Clone one, run the suite, then break something and watch which test goes red. Across all chapters the suites are 70 · 63 · 65 tests.

Remember these

Key Takeaways

- ✓ Hexagonal, onion and clean are one idea: domain at the center.
- ✓ Dependencies point inward; infrastructure depends on the domain.
- ✓ A port is a domain-owned interface; an adapter is the GoF Adapter pattern on it.
- ✓ You gain testability and swappable infrastructure — at the cost of indirection.
- ✓ It's the Dependency Inversion Principle, applied to the whole system.

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Hexagonal Architecture Explained** Cockburn & Garrido de Paz · Humans & Technology, 2024
Ports & adapters from the person who invented it — the definitive treatment.

📖 **Clean Architecture** Robert C. Martin · Prentice Hall, 2017
The dependency rule, and how these three styles are one idea seen from three angles.

📖 **Get Your Hands Dirty on Clean Architecture, 2nd ed.** Tom Hombergs · Packt, 2023
A working Java codebase — package structure, mapping, and the cost of the style.

📖 **Clean Applications with Hexagonal Architecture** Enrique Medina Montenegro · Manning MEAP, 2026
Being written now — ports and adapters carried through a whole application.

📖 **Implementing Domain-Driven Design** Vaughn Vernon · Addison-Wesley, 2013
Ch. 4 Architecture — hexagonal presented as DDD's natural host.

Hexagonal — Alistair Cockburn → alistair.cockburn.us/hexagonal-architecture

Onion — Jeffrey Palermo → jeffreypalermo.com/2008/07/the-onion-architecture-part-1

Clean — Robert C. Martin → blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

What's Next

Module 14 · with Emre Akış

Introduction to Distributed Architecture

SOA & Microservices

The baton passes: from a well-structured monolith to systems spread across many services and a network.

Ayvalık Bank — Hexagonal (HA)

The same banking system rebuilt with ports & adapters. See the domain depend only on its ports; infrastructure plugs in at the edges.

Ayvalık Bank · Hexagonal Architecture

Java github.com/kaldiroglu/AyvalikBankHA-JAVA

.NET github.com/kaldiroglu/AyvalikBankHA-NET

Python github.com/kaldiroglu/AyvalikBankHA-Python

IN THIS SESSION, LOOK AT

- domain/port , domain/service (the core)
- adapter/in/web , adapter/out/persistence (edges)

Questions?

Domain at the center. Infrastructure at the edges.